Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 9
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest



In cursul anterior

- Debugging and Performance Evaluation
- Performance Issues
- Profiling and Tracing
- Performance Counters



Cuprins

- Performance Analysis
  - Programming
  - Tools
  - Techniques
- Parallel Correctness Challenges
  - Debugging
  - Testing

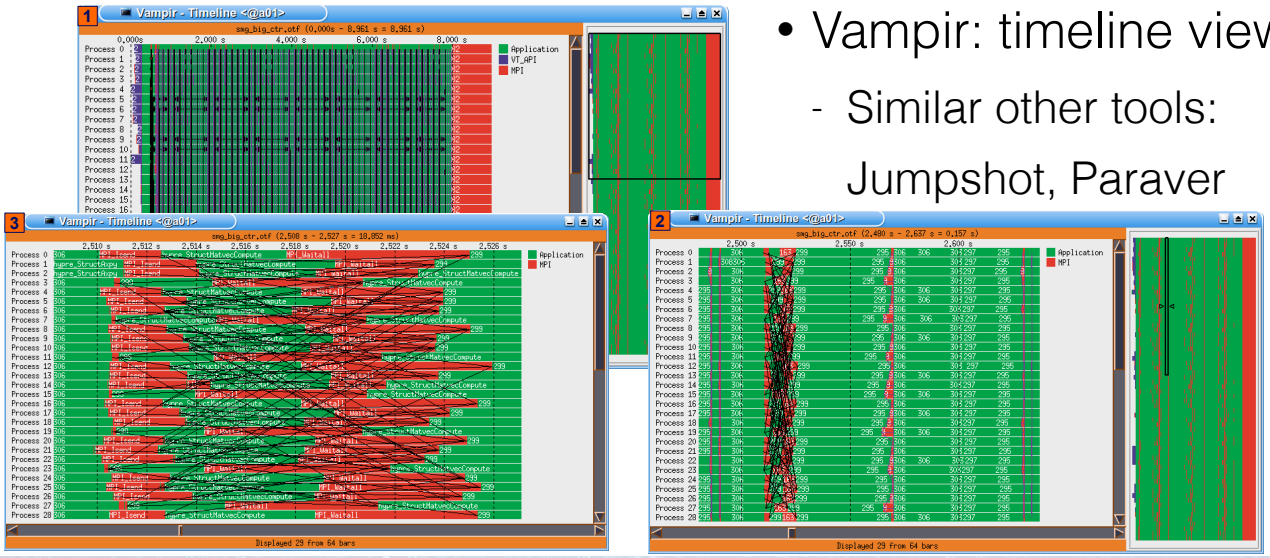


Performance Analysis - Tools & Techniques

Performance Data Analysis

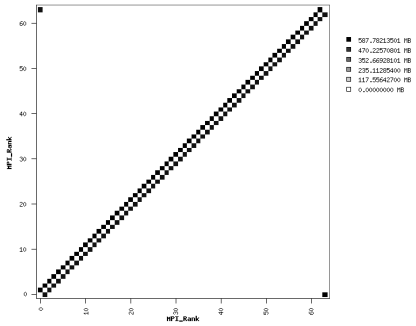
- Draw conclusions from measured performance data
- Manual analysis
  - Visualization
  - Interactive exploration
  - Statistical analysis
  - Modeling
- Automated analysis
  - Try to cope with huge amounts of performance by automation
  - Examples: Paradyn, KOJAK, Scalasca, Periscope

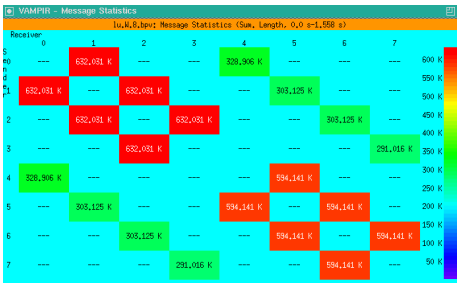
Trace File Visualization

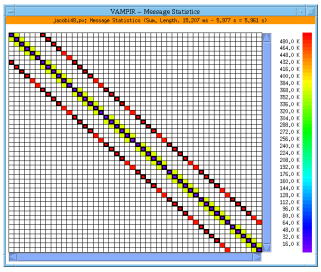


- Vampir: timeline view
  - Similar other tools: Jumpshot, Paraver

Trace File Visualization



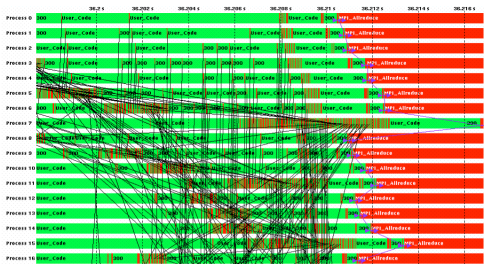




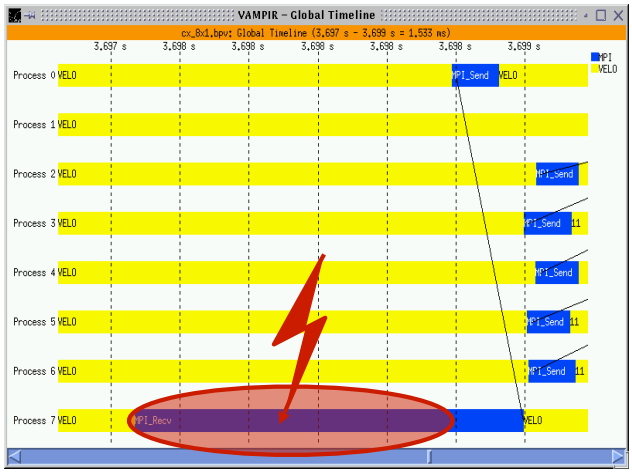
- Vampir/IPM: message communication statistics

Automated Performance Analysis

- Reason for Automation
  - Size of systems: several tens of thousand of processors
  - LLNL Sequoia: 1.6 million cores
  - Trend to multi-core
- Large amounts of performance data
  - Several gigabytes or even terabytes
- Not all programmers are performance experts
  - Scientists want to focus on their domain
  - Need to keep up with new machines
- Automation can solve some of these issues

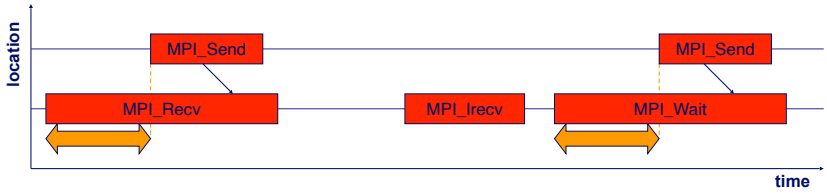


Automation Example

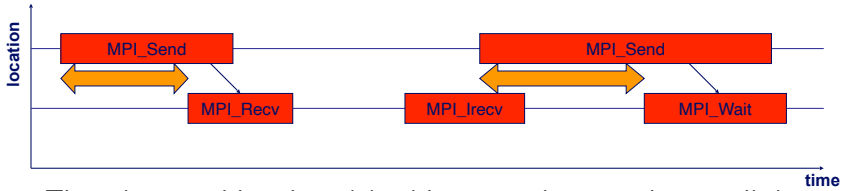


- „Late sender“ pattern
- This pattern can be detected automatically by analyzing the trace

MPI Pattern: Late Sender / Receiver

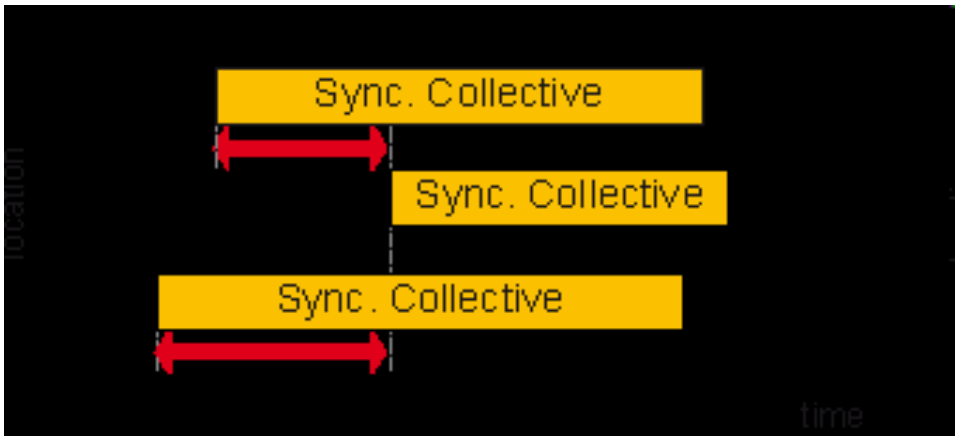


- Late Sender: Time lost waiting caused by a blocking receive operation posted earlier than the corresponding send operation



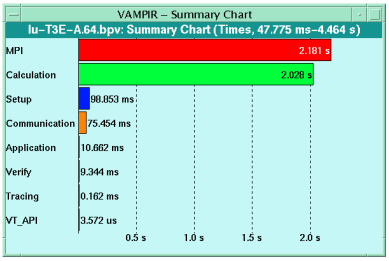
- Late Receiver: Time lost waiting in a blocking send operation until the corresponding receive operation is called

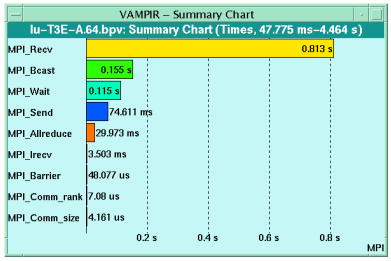
MPI Pattern: Wait at Barrier



- Time spent in front of MPI synchronizing operation such as barriers

Vampir Overview Statistics

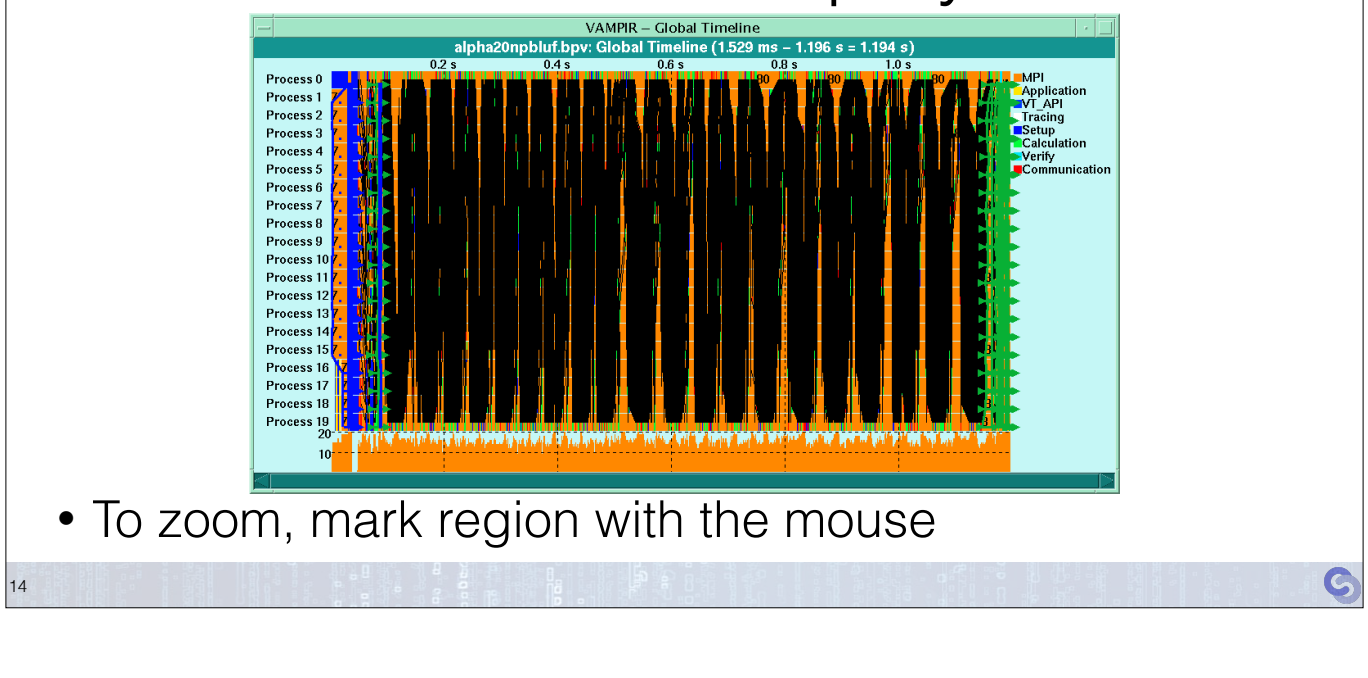




- Aggregated profiling information
  - Execution time
  - Number of calls
- This profiling information is computed from the trace
  - Change the selection in main timeline window
- Inclusive or exclusive of called routines

Timeline Display – Contents

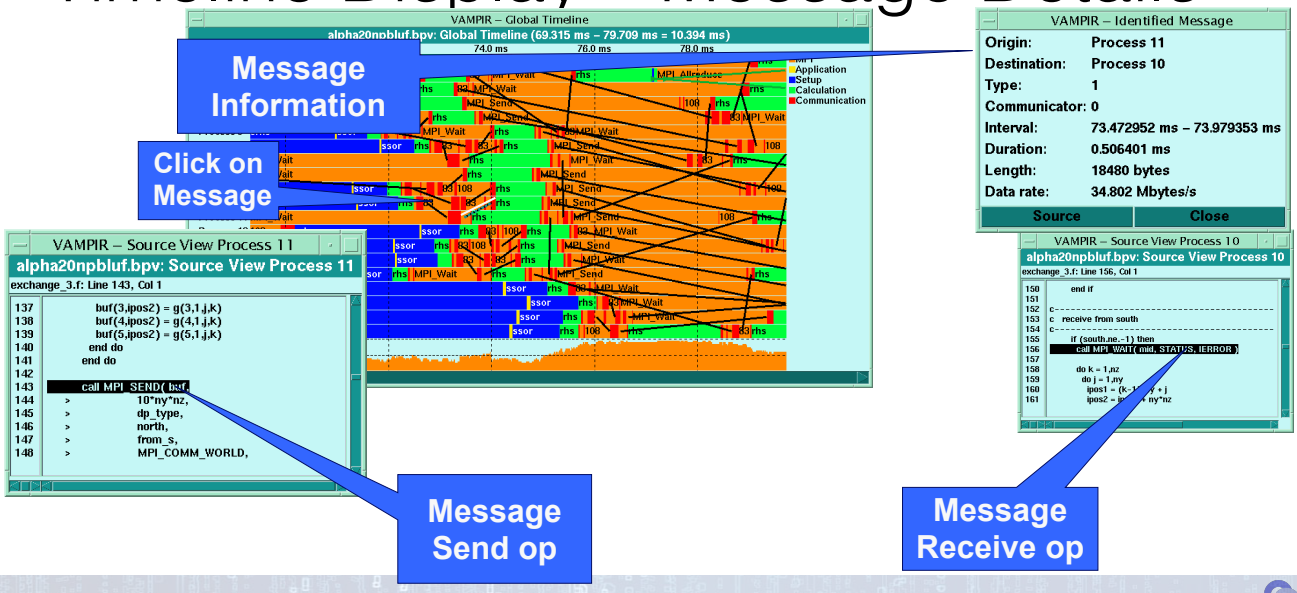
- Shows all selected processes
- Shows state changes (activity color)
- Shows messages, collective and MPI–IO operations
- Can show parallelism display at the bottom



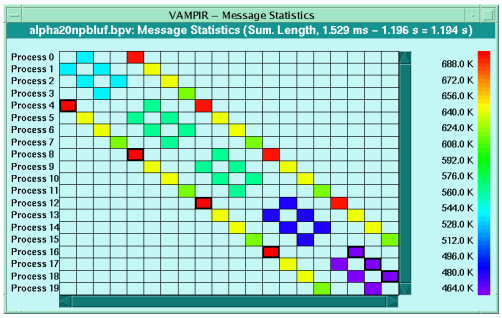
Timeline Display

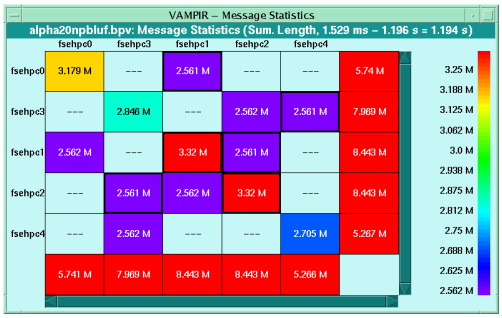
- To zoom, mark region with the mouse

Timeline Display – Message Details



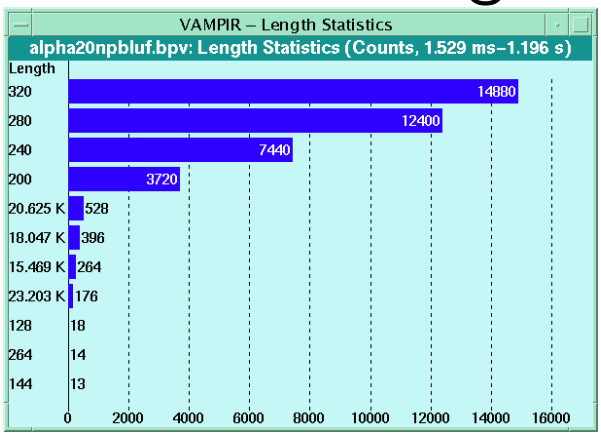
Communication Statistics

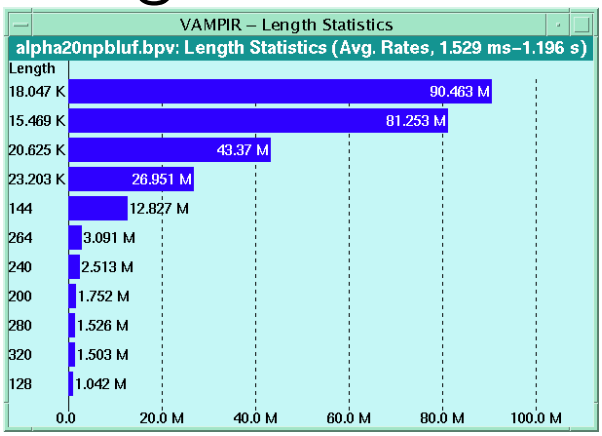




- Message statistics for each process/node pair:
  - Byte and message count
  - Min/max/avg message length, bandwidth

Message Histograms



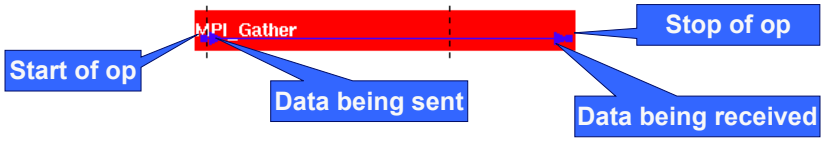


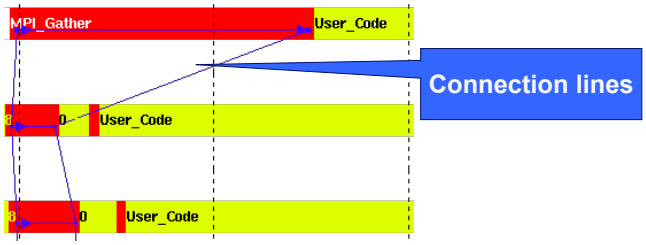
- Message statistics by length, tag or communicator
  - Byte and message count
  - Min/max/avg bandwidth



Collective Operations

- For each process: mark operation locally



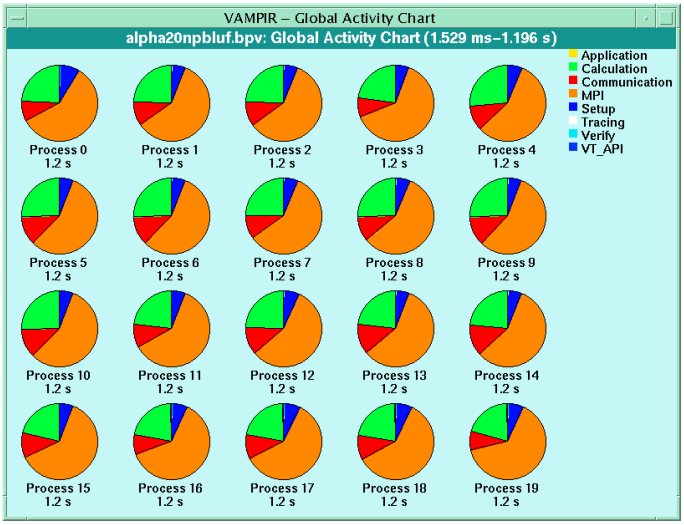


- Connect start/stop points by lines

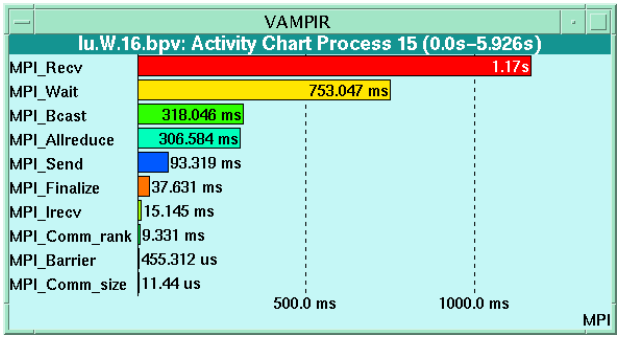


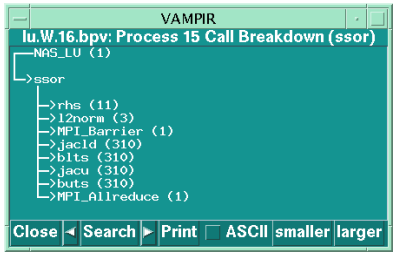
Activity Chart

- Profiling information for all processes

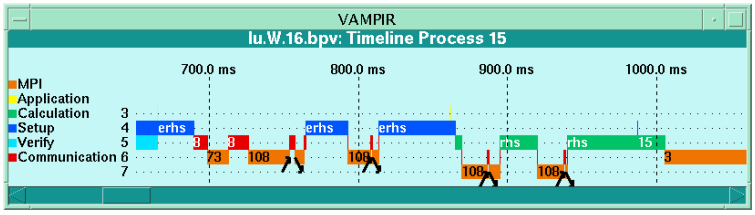


Process – Local Displays

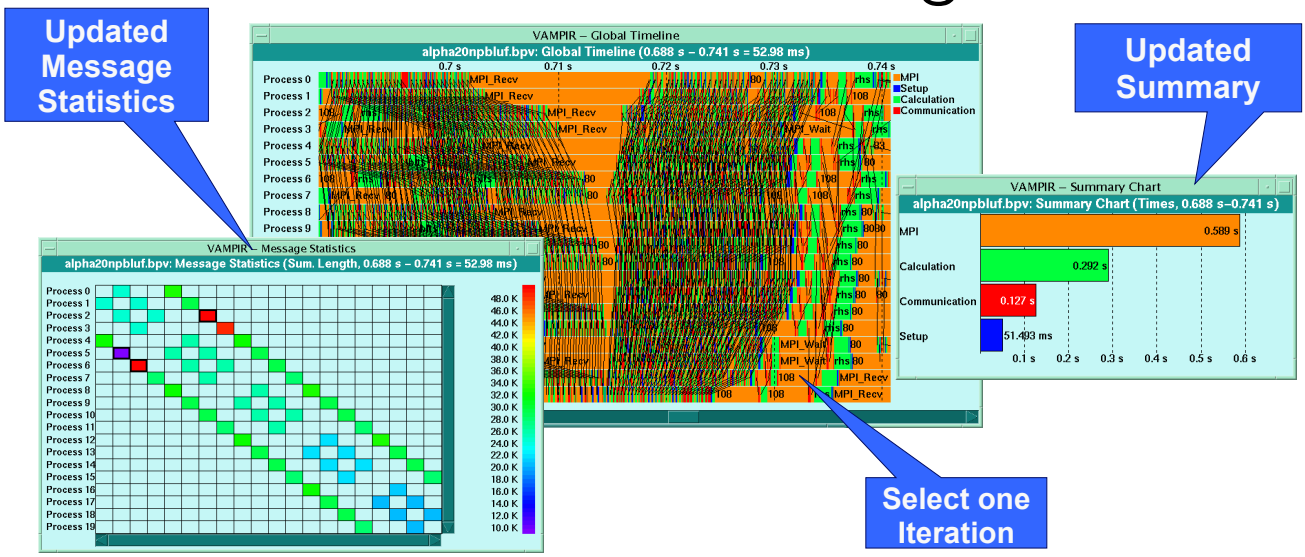




- Timeline: calling levels
- Activity chart
- Calling tree: number of calls

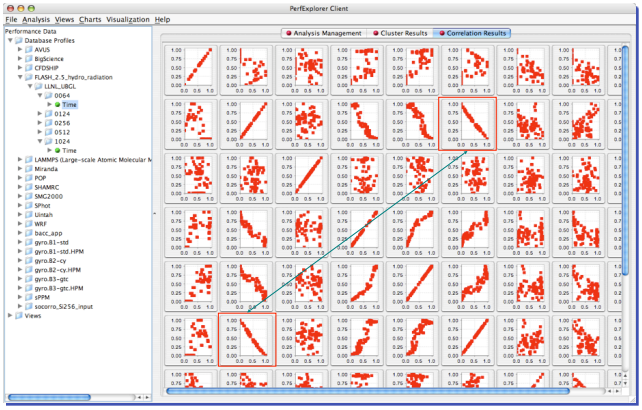


Effects of Zooming



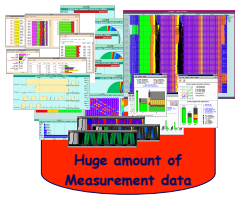
PerfExplorer – Correlation Analysis (Flash)

- Describes strength and direction of a linear relationship between two variables (events) in the data

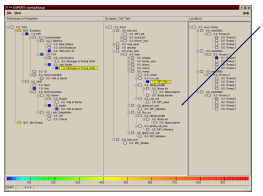


Scalasca / KOJAK – Basic Idea

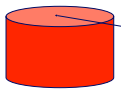
- "Traditional" Tool



- Automatic Tool



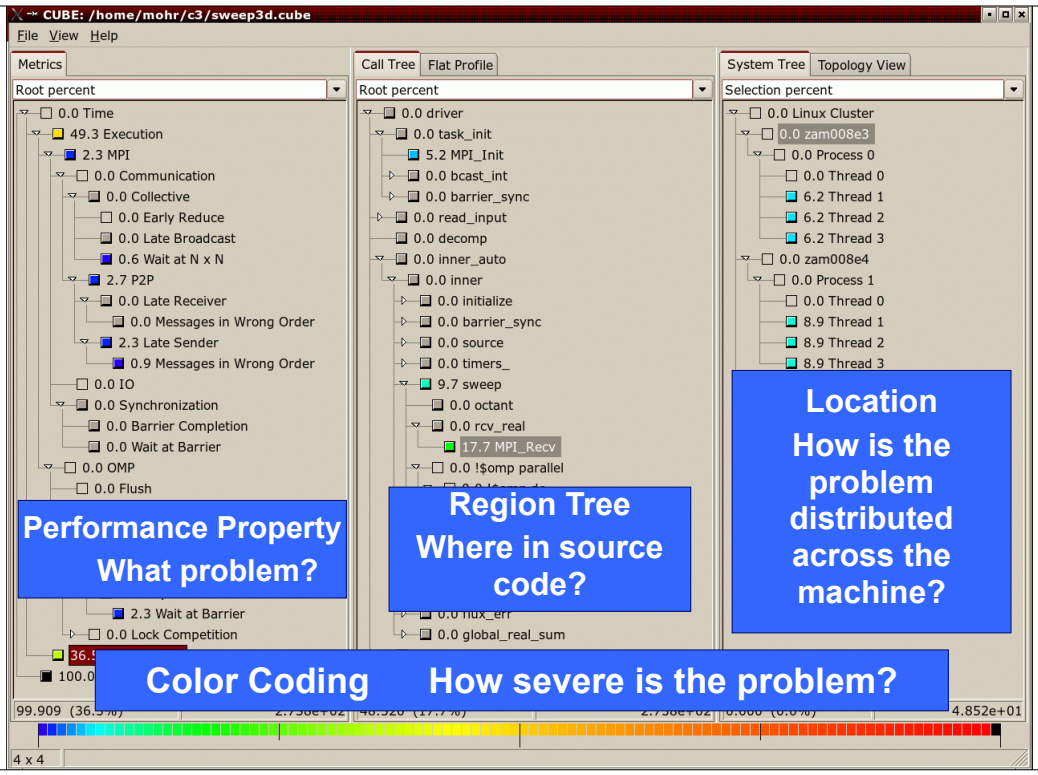
Simple:
1 screen +
2 commands +
3 panes



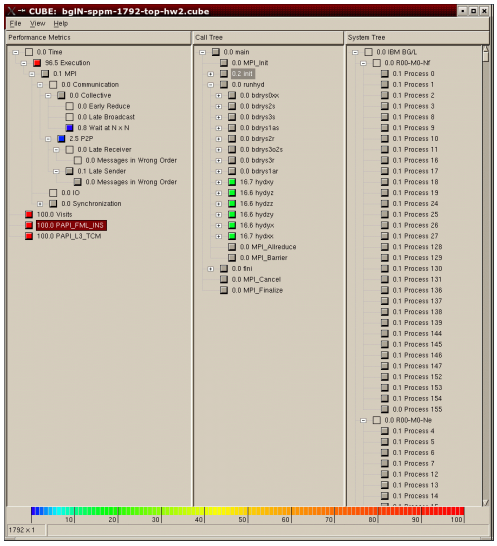
Relevant
problems
and data

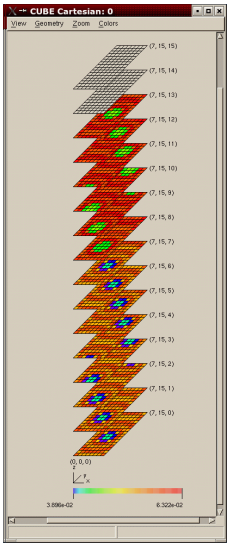
- For non-standard / tricky cases (10%)
- For expert users
- For standard cases (90% ?!)
- For "normal" users
- Starting point for experts

⇨ **More productivity for performance analysis process!**

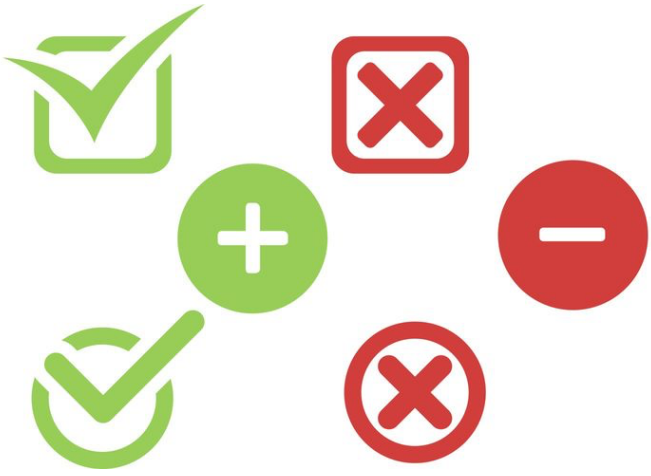


KOJAK: sPPM run on (8x16x14) 1792 PEs





- Topology display
- Shows distribution of pattern over HW topology
- Easily scales to even larger systems



Parallel Correctness Challenges

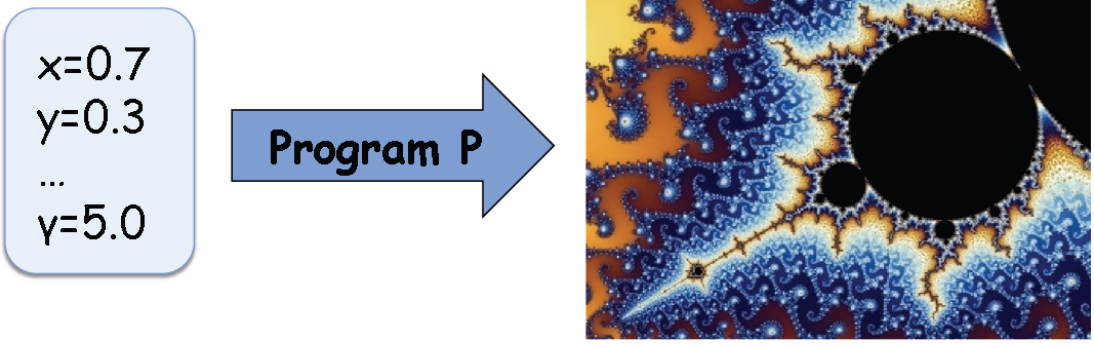
Parallel Correctness Challenges

- Parallel programming presents a number of new challenges to writing correct software
  - New kinds of bugs: data races, deadlocks, etc.
  - More difficult to test programs and find bugs
  - More difficult to reproduce errors
- Key Difficulty: Potential non-determinism
  - Order in which threads execute can change from run to run
  - Some runs are correct while others hit bugs



Parallel Correctness Challenges

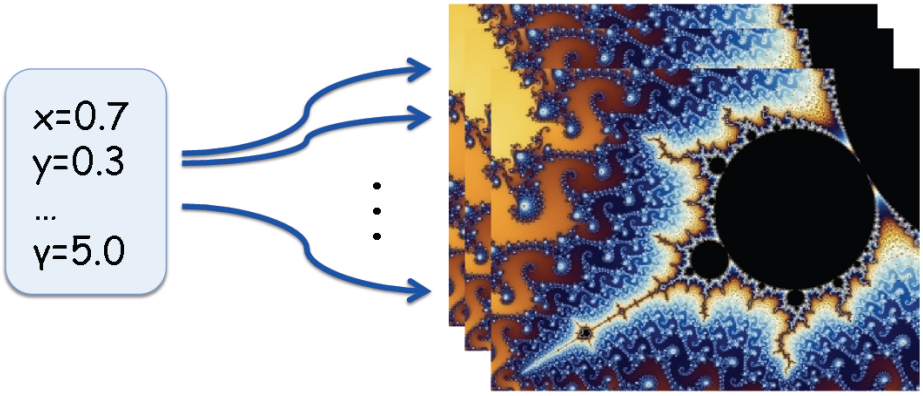
- For sequential programs, we typically expect that same input → same output





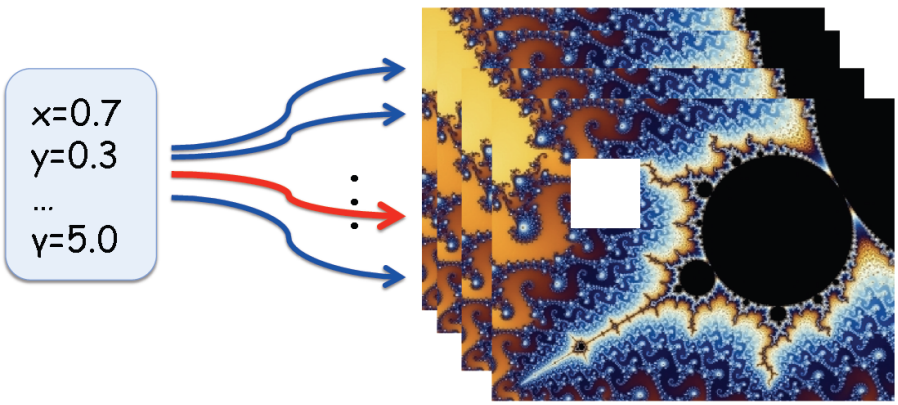
Parallel Correctness Challenges

- But for parallel programs, threads can be scheduled differently each run



Parallel Correctness Challenges

- But for parallel programs, threads can be scheduled differently each run





Parallel Correctness Challenges

- But for parallel programs, threads can be scheduled differently each run
- A bug may occur under only rare schedules
  - In 1 run in 1000 or 10,000 or …
- May occur only under some configurations:
  - Particular OS scheduler
  - When machine is under heavy load (> 80%)
  - Only when debugging/logging is turned off!



Testing Parallel Programs

- For **sequential** programs:
  - Create several test inputs with known answers.
  - Run the code on each test input.
  - If all tests give correct output, have some confidence in the program
  - Have intuition about which “edge cases” to test.
- But for **parallel** programs:
  - Each run tests only a single schedule.
  - How can we test many different schedules?
  - How confident can we be when our tests pass?



Testing Parallel Programs

- Possible Idea: Can we just run each test thousands of times?
- Problem: Often not much randomness in OS scheduling
  - May waste much effort, but test few different schedules
  - Recall: Some schedules tend to occur only under certain configurations – hardware, OS, etc
  - One easy parameter to change: load on machine



Stress Testing

- Idea: Test parallel program while oversubscribing the machine
  - On a 4-core system, run with 8 or 16 threads
  - Run several instances of the program at a time
  - Increase size to overflow cache/memory
  - Effect: Timing of threads will change, leading to
    - **Different thread schedules**
- Pro: Very simple idea, easy to implement
  - And often works!



Noise Making / Random Scheduling

- Idea: Run with random thread schedules
  - E.g., insert code like:
    - if (rand() < 0.01) usleep(100)
    - if (rand() < 0.01) yield()
  - Can add to “suspicious” or “tricky” code
  - Or use tool to seize control of thread scheduling
- Pros: Still fairly simple and often effective
  - Explores different schedules than stress testing
  - Many tools can perform this automatically



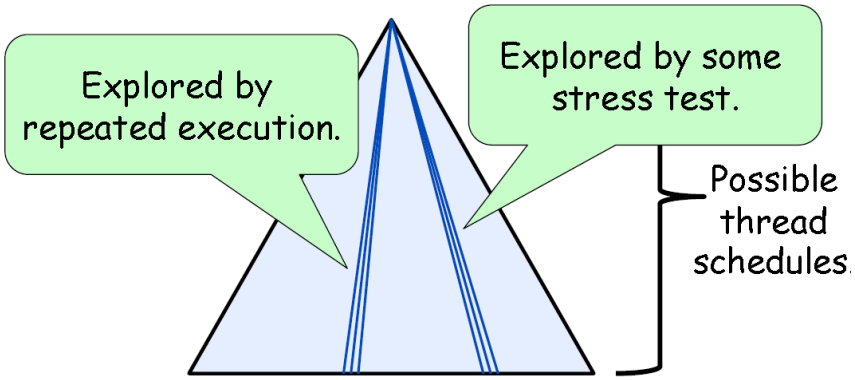
Noise Making / Random Scheduling

- IBM’s ConTest: Noise-making - Java
- Berkeley’s Thrille - C + pThreads
- CalFuzzer does simple random scheduling - Java
- Microsoft Research’s Cuzz - .NET
- Many of these tools provide replay
  - Same random number seed → same schedule



Limitations of Random Scheduling

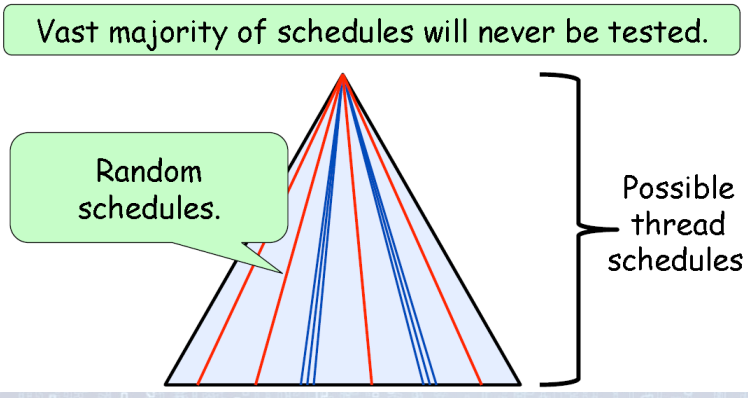
- Parallel programs have huge number of schedules – exponential in length of a run





Limitations of Random Scheduling

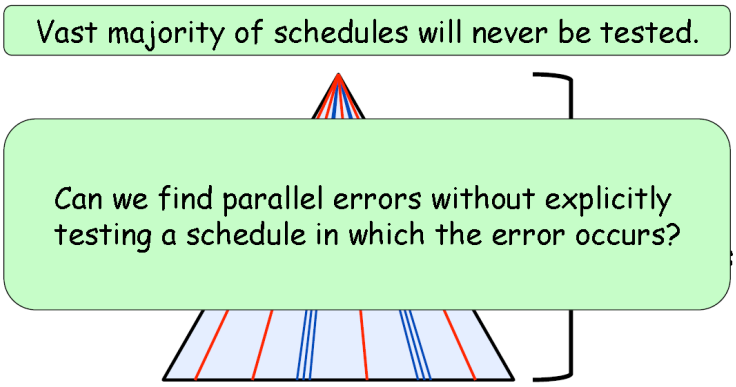
- Parallel programs have huge number of schedules – exponential in length of a run





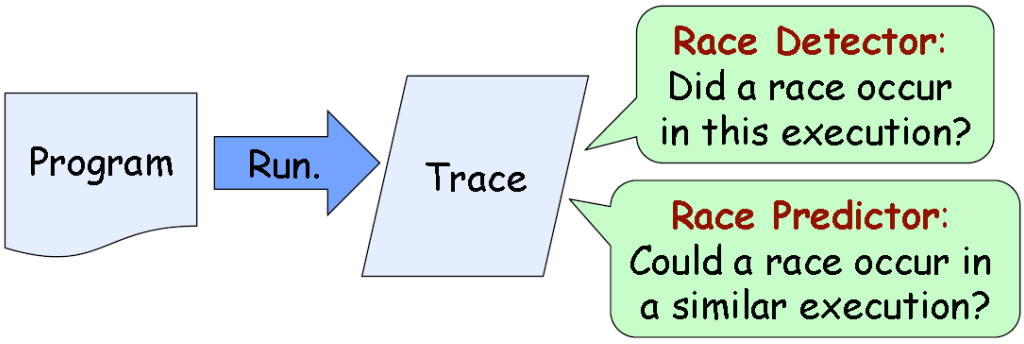
Limitations of Random Scheduling

- Parallel programs have huge number of schedules – exponential in length of a run



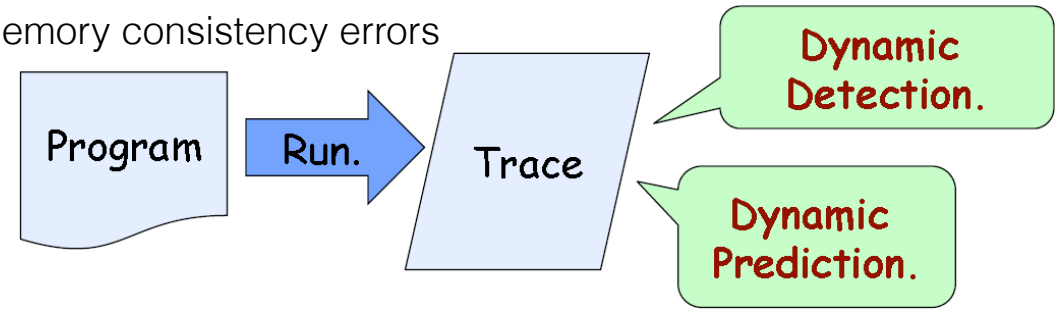
Detecting/Predicting Parallel Bugs

- Say we observe a test run of a parallel program that doesn't obviously fail
- Key Question: Can we find possible parallel bugs by examining the execution?



Detecting/Predicting Parallel Bugs

- Techniques/tools exist for:
  - Data races
  - Atomicity violations
  - Deadlocks
  - Memory consistency errors



Static Analysis

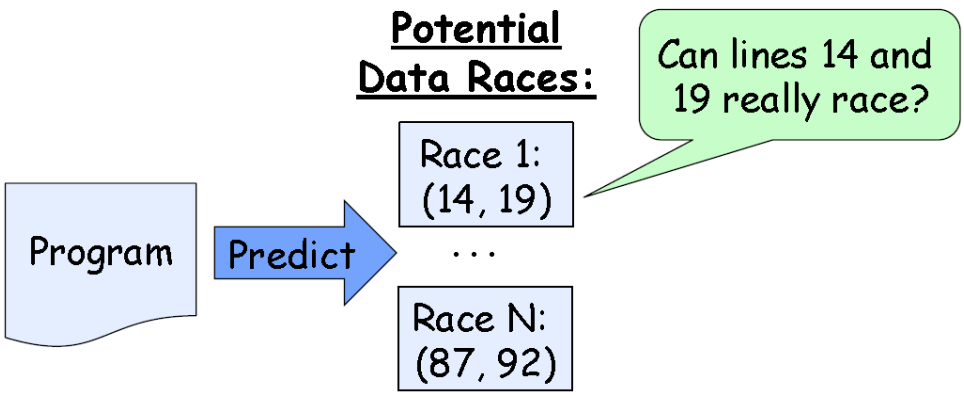
- So far we've only discussed dynamic analyses
  - Examine a real run/trace of a program
- Static analyses predict data races, deadlocks, etc., **without** running a program
  - Only examine the source code
  - Area of active research for ~20 years
  - Potentially much better coverage than dynamic analysis – examines all possible runs
  - But typically also more false warnings

Active Random Testing Overview

- **Problem**: Random testing can be very effective for parallel programs, but can miss many potential bugs
- **Problem**: Predictive analyses find many bugs, but can have false warnings
  - Time consuming and difficult to examine reported bugs and determine whether or not they are real
- **Key Idea**: Combine them – use predictive analysis to find potential bugs, then **biased** random testing to actually create each bug

Active Random Testing

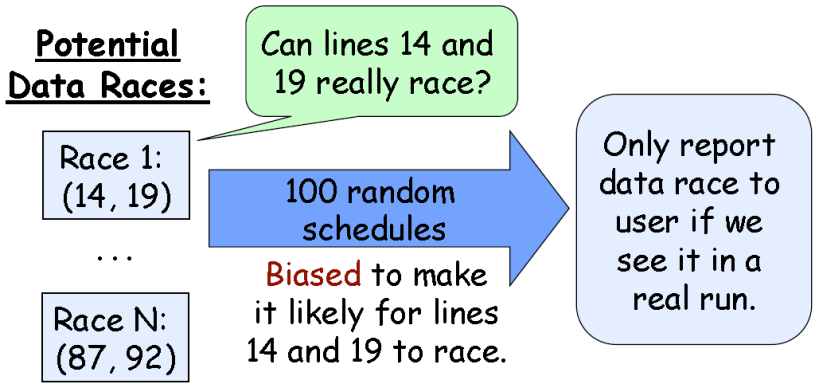
- **Key Idea**: Combine them – use predictive analysis to find potential bugs, then **biased** random testing to actually create each bug





Active Random Testing

- **Key Idea**: Combine them – use predictive analysis to find potential bugs, then **biased** random testing to actually create each bug



Active Random Testing Tools

- CalFuzzer - extensible, open-source tool for active testing of Java programs
  - Data races, atomicity bugs, and deadlocks
  - RaceFuzzer is the active testing algorithm for data races
- Thrille for C / pThreads & Data Races
- And UPC-Thrille for Unified Parallel C
  - Part of the Berkeley UPC system

Testing & Debugging Conclusions

- Many tools available right now to help find bugs in parallel software
  - Data races, atomicity violations, deadlocks
- **No silver bullet solution!**
  - Design how threads will coordinate and share/protect data
  - Tools help catching mistakes when design is not followed
  - Ad hoc parallelization likely to **never** be correct, even with help from tools



Tools References

- IBM's ConTest: Noise-making - Java
  - https://www.research.ibm.com/haifa/conferences/hvc2010/present/Testing_and_Debugging_Concurrent_Software.pdf
- Cuzz: Random scheduling for C++/.NET
  - http://research.microsoft.com/en-us/projects/cuzz/
- Intel Inspector - Locate and debug threading, memory errors in C/C++
  - https://software.intel.com/content/www/us/en/develop/tools/oneapi/components/inspector.html?wapkw=Thread%20Checker%20and%20Parallel%20Inspector#gs.0pp194
- Helgrind, DRD, ThreadSanitizer Dynamic Data Race Detection/Prediction for C/C++
  - http://valgrind.org/docs/manual/hg-manual.html
  - http://code.google.com/p/data-race-test/
- CHORD - Static Race/Deadlock Detection for Java
  - https://www.cse.msu.edu/~cse914/Overheads/mmcgill-java-static-race-detector-6per.pdf



Tools References

- CalFuzzer - Java
  - https://github.com/ksen007/calfuzzer
  - https://www.seas.upenn.edu/~mhnaik/papers/cav09.pdf
- Thrille - C
  - http://github.com/nicholasjalbert/Thrille
- CHESS - C++/.NET Model Checking, Race Detection
  - https://www.microsoft.com/en-us/research/publication/chess-a-systematic-testing-tool-for-concurrent-software/
- Java Path Finder - Model Checking for Java
  - https://blog.devgenius.io/introduction-to-java-pathfinder-80828b53309a
- Tau Performance System - Fortran, C, C++, Java, Python
  - http://www.cs.uoregon.edu/research/tau/home.php
- Scalasca Performance Analysis - C/C++ and Fortran
  - https://www.scalasca.org/
- Performance Application Programming Interface - PAPI & Exa-PAPI
  - http://icl.cs.utk.edu/exa-papi/



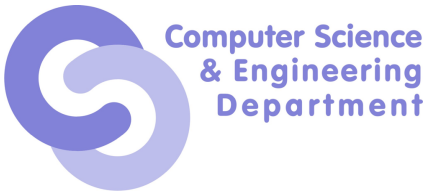
Ce am invatat in aces curs?

- Performance Analysis
  - Programming
  - Tools
  - Techniques
- Parallel Correctness Challenges
  - Debugging
  - Testing



Thank you for your attention.





emil.slusanschi@cs.pub.ro
cs.pub.ro